TAREA 5.1: AUDITORÍA DE RENDIMIENTO E INDEXACIÓN EN MYSQL

FICHA TÉCNICA DE LA ACTIVIDAD

Unidad Didáctica	Unidad 5: Optimización de consultas e índices en MySQL / MariaDB
Nombre de la Tarea	Tarea 5.1: Auditoría de Rendimiento e Indexación en Servidores de Producción
Resultado de Aprendizaje (RA)	**RA4:** Optimiza el rendimiento del sistema gestor monitorizando los accesos y ajustando los parámetros del motor.
Criterios de Evaluación (CE)	**CE 4.b:** Se han identificado los factores que afectan al rendimiento del sistema. **CE 4.c:** Se han analizado las trazas de ejecución de las consultas. **CE 4.d:** Se han creado índices y otras estructuras de optimización. **CE 4.f:** Se han reestructurado los datos para mejorar la velocidad de acceso.
Tiempo Estimado	6 horas lectivas (equivalente a 3 sesiones de clase en taller de sistemas)

MÉTODO DE ENTREGA Y FORMATO

- **[ ] Documento de Memoria:** Documento técnico en formato **PDF** (Tarea5_1_NombreAlumno.pdf) con la explicación justificada de cada paso y capturas de pantalla de la terminal de comandos. El formato debe ser impecable: texto justificado, enunciados destacados en negrita y comentarios descriptivos debajo de cada captura.
- **[ ] Archivo de Código:** Archivo de script SQL (tarea5_1_nombrealumno.sql) que contenga todas las sentencias de creación de índices, consultas y comandos de análisis, perfectamente comentados.
-

```
[ ] Verificación de Autoría: Para que las capturas
de pantalla sean válidas, debe ser visible en todo momento el
prompt de tu sistema operativo con tu usuario de Linux
(usuario@instancia-openstack:~$) o se debe incluir al inicio de cada
bloque de consultas SQL un comentario identificativo de la forma:
SELECT 'Autor: Nombre Completo del Alumno - Tarea 5.1' AS
identificador;
```

1. DESCRIPCIÓN GENERAL. OBJETIVOS

El objetivo fundamental de esta tarea es que el alumno actúe como un Administrador de Bases de Datos (DBA) junior que asume la responsabilidad de auditar y optimizar el rendimiento de un servidor MySQL de desarrollo. Utilizando la base de datos empresarial **jardineria**, se deberán detectar ineficiencias críticas de acceso a disco (barridos secuenciales completos de tablas o *Full Table Scans*), resolver problemas de pérdida de sargabilidad por malas prácticas de desarrollo, evaluar e implementar indexaciones compuestas aplicando de manera estricta la regla del prefijo izquierdo, y estructurar búsquedas semánticas eficientes mediante índices de texto completo (FULLTEXT).

2. CONSIDERACIONES PREVIAS

- Se requiere tener acceso a una instancia de MySQL Server (v8.0+ o MariaDB v10.5+) en funcionamiento sobre Ubuntu Server.
- La base de datos de producción **jardineria** debe estar completamente importada, con datos de pruebas consistentes.
- Todas las operaciones de consulta y monitorización se realizarán directamente desde la interfaz de línea de comandos de MySQL (*MySQL Client*) en la terminal SSH, evitando interfaces gráficas para emular el flujo de trabajo de administración de sistemas sobre servidores remotos desatendidos.

3. ACTIVIDADES A REALIZAR

Ejercicio 1: Inspección Inicial de Estructuras de Indexación (1 Punto)

Consulte de manera exhaustiva cuáles son los índices físicos activos en la tabla producto utilizando las instrucciones nativas del diccionario de datos de MySQL. Documente la salida y explique detalladamente:

1. Qué claves e índices ha identificado.
2. Qué significa técnicamente el valor devuelto en la columna Non_unique para cada uno de ellos.

3. Qué tipo de estructura física de datos (Index_type) utiliza el motor InnoDB para gestionarlos de forma predeterminada.

Ejercicio 2: Auditoría Comparativa de Planes de Ejecución Básicos (1 Punto)

Haga uso de la sentencia de diagnóstico EXPLAIN para obtener y analizar el plan de ejecución de las dos consultas siguientes:

```
-- Consulta A
SELECT * FROM producto WHERE codigo_producto = 'OR-114';

-- Consulta B
SELECT * FROM producto WHERE nombre = 'Evonimus Pulchellus';
```

Responda razonadamente a las siguientes cuestiones analizando la salida del optimizador:

1. ¿Cuál de las dos consultas realizará un menor número de comparaciones de registros y accesos a disco físico para hallar el resultado?
2. Indique de manera numérica cuántas comparaciones de filas estima el optimizador (rows) para cada caso.
3. Justifique la causa técnica de esta diferencia analizando los valores devueltos en las columnas type y key del EXPLAIN.

Ejercicio 3: Pérdida de Sargabilidad por Funciones de Fecha (1 Punto)

Suponga que su equipo de desarrollo web ha propuesto las siguientes alternativas para calcular el promedio de facturación total del año 2008 en la tabla pago:

```
-- Alternativa A (Inhabilitando el árbol B-Tree)
SELECT AVG(total) FROM pago WHERE YEAR(fecha_pago) = 2008;

-- Alternativa B (Sargable en rango puro)
SELECT AVG(total) FROM pago WHERE fecha_pago >= '2008-01-01' AND fecha_pago <= '2008-12-31';
```

Asumiendo que existe un índice activo sobre la columna fecha_pago, ejecute un análisis EXPLAIN sobre ambas alternativas y responda de forma técnica:

1. ¿Por qué el optimizador de costes de MySQL descarta el uso del índice en la Alternativa A devolviendo un acceso de tipo ALL? Explique qué ocurre en la CPU del servidor.
2. ¿Cómo cambia el método de acceso de la columna type en la Alternativa B y por qué se considera una práctica de "diseño sargable"?

Ejercicio 4: Optimización de Consultas Multitabla con Relación INNER JOIN (1 Punto)

Se desea optimizar la velocidad de respuesta de la siguiente consulta de facturación e histórico de pedidos:

```
SELECT * FROM cliente INNER JOIN pedido
ON cliente.codigo_cliente = pedido.codigo_cliente
WHERE cliente.nombre_cliente LIKE 'A%';
```

1. Ejecute un EXPLAIN previo a la optimización para comprobar el número de filas estimadas (rows) y el tipo de acceso (type) en ambas tablas.
2. Diseñe e implemente el índice físico más adecuado para acelerar la restricción del bloque WHERE.
3. Vuelva a analizar la traza de la consulta con EXPLAIN y argumente el impacto en el rendimiento contrastando las filas analizadas antes y después.

Ejercicio 5: El Límite de los Índices Estándar ante Comodines Iniciales (1 Punto)

Analice las siguientes consultas multitabla requeridas por el software de administración:

```
-- Consulta de Patronaje Intermedio
SELECT * FROM cliente INNER JOIN pedido
ON cliente.codigo_cliente = pedido.codigo_cliente
WHERE cliente.nombre_cliente LIKE '%A%';

-- Consulta de Patronaje Final
SELECT * FROM cliente INNER JOIN pedido
ON cliente.codigo_cliente = pedido.codigo_cliente
WHERE cliente.nombre_cliente LIKE '%A';
```

1. Justifique detalladamente por qué no es posible optimizar el tiempo de ejecución ni el número de lecturas en disco de estas sentencias, incluso si existe un índice B-Tree creado sobre la columna nombre_cliente.
2. Explique de qué manera navega el optimizador por la estructura física del árbol en estos escenarios específicos y por qué acaba determinando realizar un Full Table Scan.

Ejercicio 6: Implementación de Índices Invertidos para Búsquedas Semánticas (1 Punto)

Cree un índice de tipo FULLTEXT compuesto de forma síncrona sobre las columnas nombre y descripcion de la tabla producto de la base de datos jardineria.

1. Proporcione la sentencia DDL exacta de alteración estructural (ALTER TABLE).
2. Escriba la sintaxis de consulta SQL optimizada utilizando las funciones nativas MATCH() y AGAINST() para buscar el token 'acero'.
3. Verifique con EXPLAIN que el método de acceso cambia a fulltext y argumente la ganancia en el uso de I/O de disco frente a una búsqueda basada en operador LIKE convencional.

Ejercicio 7: Modos de Explotación de Índices FULLTEXT (1 Punto)

Haciendo uso del índice de texto completo creado en el Ejercicio 6, realice las siguientes consultas utilizando la sintaxis formal de MySQL y documente la diferencia en los conjuntos de datos obtenidos:

1. **Modo Lenguaje Natural:** Busque los productos que contengan la palabra 'planta' en modo por defecto (IN NATURAL LANGUAGE MODE).
2. **Modo Booleano Restrictivo:** Busque los registros que contengan obligatoriamente la palabra 'tronco' pero que no incluyan la palabra 'árbol' bajo ningún concepto.
3. **Modo Truncamiento / Comodín de Derecha:** Busque todas las variantes léxicas que comiencen por el prefijo 'planta' (por ejemplo: "planta", "plantas", "plantón", etc.).
4. **Búsqueda de Frase Literal:** Busque los registros que contengan la secuencia secuencial exacta de caracteres "proviene de las costas".

Ejercicio 8: Creación de Índices de Columnas Múltiples (Compuestos) (1 Punto)

Escriba la sentencia DDL para crear un índice secundario no único compuesto por las columnas apellido_contacto y nombre_contacto pertenecientes a la tabla cliente. El índice físico resultante debe llamarse estrictamente idx_apellido_nombre.

Ejercicio 9: Auditoría Analítica de la Regla del Prefijo Izquierdo (1 Punto)

Ejecute la sentencia diagnóstica EXPLAIN para auditar la eficiencia del índice compuesto idx_apellido_nombre creado en el ejercicio anterior sobre los siguientes tres escenarios de búsqueda requeridos por el servicio técnico:

- **Escenario A (Par Completo):** Buscar el cliente con apellido 'Villar' y nombre 'Javier'.
- **Escenario B (Prefijo Izquierdo Activo):** Buscar el cliente utilizando únicamente el apellido 'Villar'.
- **Escenario C (Prefijo Izquierdo Ausente):** Buscar el cliente utilizando únicamente el nombre 'Javier'.

Analice de forma crítica el plan de ejecución de cada uno de los tres escenarios. Responda:

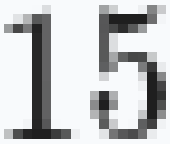
1. ¿Cuántas filas físicas estima examinar el motor en cada caso?
2. ¿En qué escenario se descarta por completo el uso del índice compuesto, forzando un barrido completo de la tabla? Justifique el motivo técnico basándose en la física de almacenamiento del árbol B+.

Ejercicio 10: Metodología Práctica de Análisis de Prefijos de Columna (1 Punto)

Como administrador de bases de datos, debe optimizar la columna nombre_cliente de la tabla cliente (definida originalmente como un campo de texto VARCHAR de tamaño considerable). Para ahorrar consumo de memoria RAM en el buffer de índices del SGBD, desea crear un **Índice de Prefijo de Columna** óptimo.

Realice los siguientes pasos analíticos en su servidor de pruebas:

1. **Cálculo de la Cardinalidad de Base:** Obtenga el número exacto de valores únicos globales contenidos en esa columna utilizando la combinación de funciones de agrupación COUNT(DISTINCT ...).

2. **Análisis de Ganancia por Longitud de Prefijo:** Ejecute una única consulta SQL que calcule la cardinalidad de valores únicos simulando prefijos con longitudes de $N = 3, 5, 8, 11$ y  caracteres utilizando la función LEFT().
3. **Selección Técnica del Prefijo:** Identifique numéricamente el menor tamaño de prefijo que proporcione un ratio de discriminación extremadamente cercano al valor de la cardinalidad global de la columna sin prefijo.
4. **Aplicación DDL:** Cree de forma definitiva el índice de prefijo en caliente utilizando la longitud  seleccionada justificadamente en el paso anterior.

4. CALIFICACIÓN

⚠️ **IMPORTANTE:** El retraso en el plazo de entrega a través de la plataforma, la entrega del documento en formatos editables no permitidos (como .docx), capturas con identificación del alumno/a o la omisión de los enlaces de las fuentes documentales utilizadas penalizará gravemente la nota o invalidará la práctica.

RÚBRICA DE EVALUACIÓN TÉCNICA

Criterio de Evaluación	Excelente (100%)	Notable (75%)	Suficiente (50%)	Insuficiente (0%)	Peso
Contenido Técnico y Funcionalidad	Todos los comandos SQL se ejecutan de manera impecable y las	Se ejecutan todos los comandos SQL con errores	Faltan comandos o algunos índices no se han	No se resuelven las consultas ni se crean las estructuras de	**50%**

Criterio de Evaluación	Excelente (100%)	Notable (75%)	Suficiente (50%)	Insuficiente (0%)	Peso
	explicaciones justifican con absoluta precisión el comportamiento físico de los árboles de datos y del optimizador de MySQL.	mínimos de sintaxis. Se comprenden los conceptos de optimización, aunque falta profundidad en el análisis de costes.	creado correctamente. Las explicaciones técnicas de los planes de ejecución son débiles.	optimización solicitadas.	
Análisis de Procedimientos e I/O	Documenta cada cambio con trazas de EXPLAIN contrastando minuciosamente las diferencias de coste (rows, type, key). Propone mejoras sargables justificadas.	Muestra el uso de EXPLAIN pero el contraste de rendimiento se limita a listar valores sin relacionarlos directamente con el consumo de recursos de hardware.	Sigue los pasos de manera mecánica sin comprender por qué mejora o empeora la ejecución de las consultas.	No incluye trazas de análisis de planes de ejecución o estas no corresponden a sus bases de datos.	**40%**
Calidad de Documentación y Capturas	**Documento impecable (PDF).** Las capturas son perfectamente legibles, se identifica de manera inequívoca la autoría del alumno mediante el prompt y la IP del servidor. Texto justificado y estructurado con comentarios debajo de cada captura.	Documento bien estructurado en PDF. Las capturas son legibles pero falta de forma básica la identificación de autoría en alguna de ellas.	Desorganizado o sin formato adecuado. Dificultad para asociar los ejercicios con el script entregado. El formato no es PDF.	Entrega ilegible, sin capturas, o no se puede verificar de ningún modo que el alumno haya ejecutado las prácticas en su servidor.	**10%**

CRITERIO ESPECÍFICO DE DOCUMENTACIÓN Y CAPTURAS (1.5 Puntos)

Criterio	Realizado correctamente (100% - 1.5 Ptos)	Realizado con algún error (50% - 0.75 Ptos)	No realizado (0% - 0 Ptos)
Legibilidad e Identificación del Autor	Todas las capturas de terminal muestran claramente la IP de la instancia remota y el nombre de usuario del alumno en el shell, o bien el identificador mediante consulta comentada de manera consistente.	Las capturas carecen de la IP del servidor en algunas fases o la identificación no se incluye en todas las ventanas de consulta, comprometiendo la trazabilidad completa.	Capturas genéricas sin ningún elemento que identifique la procedencia del entorno de ejecución o el autor.